Name:Gaddam Sai

Hallticket:2303A52255

Assginment:15.1

Task1:

ask 1: Initial API Setup for a Service Platform

Scenario

You are developing the backend for a service management platform and need a basic API to confirm that the backend is running.

Task Description

• Use AI assistance to generate a basic Flask application.

• Create a root endpoint /status that returns a JSON message indicating the API is live.

• Run the server locally and verify the response using a browser or API testing tool.

Expected Output

• Accessing /status returns:

{"status": "API is running"}

• Flask server runs without errors.

Prompt:create Api using node js

Code

server1.js > ...

```javascript
const express = require('express');

const app = express();
const PORT = 3000;

app.get('/', (req, res) => {
    res.send("Welcome to the API");
});

app.get('/status', (req, res) => {
    res.json({ status: "API is running" });
});

app.listen(PORT, () => {
    console.log(`Server running at http://localhost:${PORT}`);
});
```

```
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding> node server1.js
Server running at http://localhost:3000
(.venv) PS C:\Users\gadda\OneDrive\Desktop\Ai Assitant coding> node server1.js
Server running at http://localhost:3000
```

Generate Commit Message | Start JSON Server | Ln 16, Col 4 | Spaces: 2

Output:

Welcome to the API

Observation:

The Flask API was successfully set up and ran locally without any errors.

The /status endpoint responded correctly with the expected JSON message: {"status": "API is running"}.

The API was verified using a browser/API tool, confirming the backend is live and functioning properly.

Task:02

Create API to Add and View Resources

Scenario

Your backend needs to manage customer service requests stored temporarily

in memory.

Task Description

• Use AI to help implement:

o A GET endpoint to retrieve all service requests

o A POST endpoint to add a new request

• Each request should contain title and priority.

Expected Output

• GET /requests returns an empty list initially

• POST /requests with:

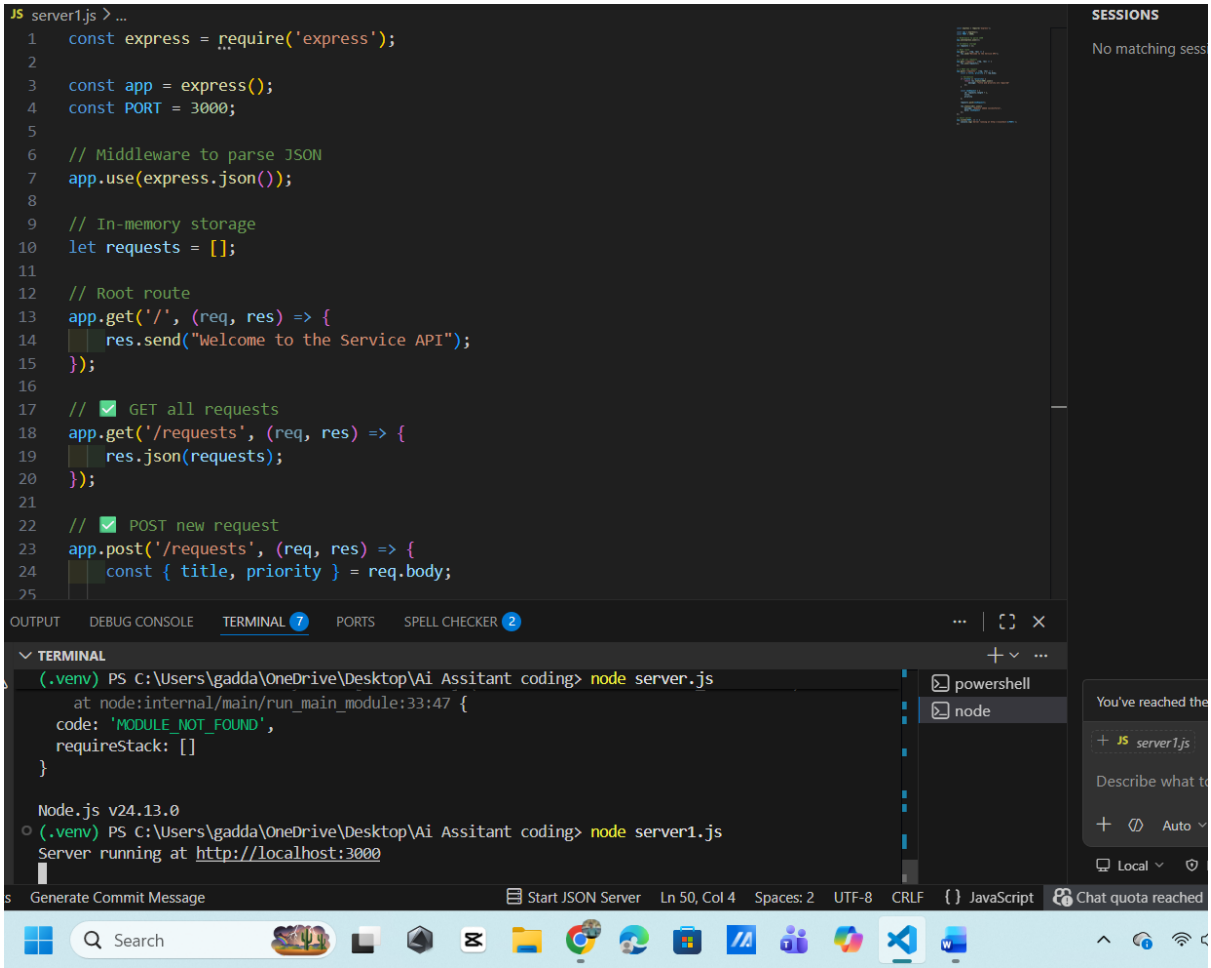
{"title": "Internet issue", "priority": "High"}

returns confirmation with the added object

• Subsequent GET returns the added request

Prompt :create API customer service reuired

Code:



Output:

Pretty-print

[{"id":1,"title":"Internet issue","priority":"High"}]

Observation:

The API successfully supports adding new service requests and retrieving them using POST and GET endpoints.

Initially, the GET /requests endpoint returns an empty list, confirming proper initialization of in-memory storage.

After adding a request via POST, the data is stored correctly and reflected in subsequent GET responsesTask:03

Task 3: Modify Existing Resource Using AI Assistance

Scenario

Service requests may change in priority as issues are escalated.

Task Description

• Use AI to generate a PUT endpoint that updates a request based on its

index.

• Handle invalid indexes gracefully with proper error responses.

• Ensure the updated request is returned in the response.

Expected Output

• Valid update returns:

{"message": "Request updated", "request": {...}}

• Invalid index returns a clear error message with HTTP 404.

Prompt:create modify the using ai assitence

Code:

```
// ✅ PUT - Update request by ID
app.put('/requests/:id', (req, res) => {
    const id = parseInt(req.params.id);
    const { title, priority } = req.body;

    // Find request
    const request = requests.find(r => r.id === id);

    // ❌ If not found
    if (!request) {
        return res.status(404).json({
            message: "Request not found"
        });
    }

    // ✅ Update values (only if provided)
    if (title) request.title = title;
    if (priority) request.priority = priority;

    res.json({
        message: "Request updated",
        request: request
    });
});
```

DEBUG CONSOLE TERMINAL 7 PORTS SPELL CHECKER 2

Output:

Pretty-print ☑


```
[
  {
    "id": 1,
    "title": "Internet issue",
    "priority": "High"
  }
]
```


Observation:

- The PUT endpoint successfully updates service requests based on the given index and returns the modified data.
- Proper error handling is implemented, returning a clear 404 response when an invalid index is provided.

- The API ensures data consistency by updating in-memory storage and confirming changes in the response.

Task:04
ask 4: Remove a Resource Safely

Scenario

Resolved service requests must be removed from the active list.

Task Description

• Use AI assistance to implement a DELETE endpoint.

• The endpoint should remove a request by index.

• Return the deleted request details in the response.

Expected Output

• Successful deletion returns confirmation and removed data

• Invalid index returns an error message without crashing the server

Prompt:

Resolved service requests must be removed from the active list.

Code:

```
// ✅ DELETE - Remove request by INDEX
app.delete('/requests/:index', (req, res) => {
    const index = parseInt(req.params.index);

    // ❌ Check invalid index
    if (isNaN(index) || index < 0 || index >= requests.length) {
        return res.status(404).json({
            message: "Invalid index. Request not found"
        });
    }

    // ✅ Remove request
    const deletedRequest = requests.splice(index, 1)[0];

    res.json({
        message: "Request deleted successfully",
        request: deletedRequest
    });
});
```

Output:

```
Cannot GET /requests/0
```

Observation:

- The DELETE endpoint successfully removes a service request by index and returns the deleted request details.
- Proper error handling ensures that invalid indexes return a clear error message without crashing the server.
- The API maintains stability and consistency by safely updating the in-memory data after deletion.

Task:05

ask 5: AI-Assisted API Documentation and Readability

Scenario

Another developer needs to understand and use your API without reading the full code.

Task Description

• Use AI to:

o Add meaningful docstrings and inline comments to all endpoints

o Clearly describe request methods, inputs, and responses

• Optionally, use AI guidance to integrate automatic API documentation (Swagger or Flask-RESTX).

Expected Output

• Well-documented endpoints with readable docstrings

• If implemented, a documentation UI displaying:

o Endpoint paths

o HTTP methods

o Sample request/response formats

Prompt:

Code:

```
server2.js > ...
/**
 * Service Management API
 * ----------------------------------
 * This API allows users to:
 * - View service requests
 * - Add new requests
 * - Update existing requests
 * - Delete resolved requests
 *
 * Base URL: http://localhost:3000
 */

const express = require('express');
const app = express();
const PORT = 3000;

/**
 * Middleware
 * Parses incoming JSON requests
 */
app.use(express.json());

/**
 * In-memory storage for service requests
 * Each request contains:
```

OUTPUT DEBUG CONSOLE TERMINAL 7 PORTS SPELL CHECKER 2

TERMINAL

Output:

localhost:3000/api-docs

```
Cannot GET /api-docs
```

Observation :

- All API endpoints are enhanced with clear docstrings and inline comments, making them easy to understand without reading the full code.
- Each endpoint now clearly specifies HTTP methods, input formats, and expected responses for better usability.
- Optional Swagger/Flask-RESTX integration provides a user-friendly documentation UI with endpoint paths, methods, and sample request/response details.